

EzManager

Security Review



24 May, 2026

Prepared by: Valves Security

Lead Auditors: Vesko210 | Merulez99 | Playboi.eth

Contents

1	About Valves Security	3
2	Disclaimer	3
3	Risk Classification	3
4	Protocol Summary	4
5	Executive Summary	4
6	Findings	5
	Medium Findings	5
	[M-01] Earned fee protocol fee computed from swap output lets callers reduce protocol revenue in no-rebalance compounds	5
	Low/Info Findings	6
	[L-01] Hostile ERC721 receiver can DoS emergency returnNft batches	6
	[L-02] changeRange accepts identical ticks, charging bot fee and slippage for a no-op rebalance	7
	[L-03] Referral fees become permanently stuck if referrer is USDC-blacklisted	7
	[L-04] Renounced ownership creates irremovable permanent pause authority	8
	[L-05] Contracts Use Single-Step Ownable Instead of Ownable2Step	9
	[I-01] storeReferrer and resolveAndStoreReferrer are duplicate functions	9
	[I-02] Dead Self-Referrer Checks in CLManager	10
	[I-03] All 10 CLManager admin setter functions emit no events	10
	[I-04] validateActionReferrer Return Value Ignored in copyPosition	11

1 About Valves Security

Valves Security is a Web3 security team founded by Veselin and Valeri. With deep experience across EVM-based protocols, Cosmos (Go) chains, and native Rust systems, we help projects identify and eliminate critical vulnerabilities. With a strong background and a strong team of freelancers we can assure a top quality audit.

Book a Security Review at valvesecurity.com or message us on X Valves Security

2 Disclaimer

The Valves team makes all effort to find as many vulnerabilities in the code in the given time period. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the implementation. We stand by our process, yet encourage follow-up audits and meaningful bug bounty incentives for continued whitehat protection.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- High - leads to a significant material loss of assets in the protocol or a group of users
- Medium - leads to a moderate material loss of assets in the protocol, moderately harms a group of users or to some disruption of the protocol's functionality
- Low - No funds at risk

Likelihood

- High - attack path is possible with reasonable assumptions. The issue is almost certain to happen.
- Medium - only a conditionally incentivized attack vector, but still relatively likely

- Low - has unlikely assumptions or requires a significant stake by the attacker

4 Protocol Summary

EZManager is a DeFi liquidity management protocol that automates and simplifies concentrated liquidity positions across decentralized exchanges. By streamlining range management, rebalancing, and position optimization, it enables users to manage on-chain liquidity more efficiently through a unified interface. It tracks each position from open to close, through every rebalance, capital change, compound and collect.

5 Executive Summary

Overview

Project	EzManager
Commit Hash	b777b72ab75fb704229aa26957a67080061bc6fc
Timeline	16/05/2026 – 23/05/2026

Scope

Id	Files in scope
1	src/CLManager
2	src/ReferralManager
3	src/EZWrapper
4	libraries/CLManagerUtils

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	1
Low/Info Risk	9

6 Findings

Medium Findings

[M-01] Earned fee protocol fee computed from swap output lets callers reduce protocol revenue in no-rebalance compounds

Fixed

Impact: Medium

Likelihood: Medium

Description

When earned fees are collected via `compoundFees`, the protocol's cut is calculated from the actual swap output (`paidUSDC`) rather than the TWAP-valued amount (`initialUSDCIn`):

```
1 // CLManagerUtils.sol in _payEarnedFeesFromCollectedTokens
2 protocolFeeUSDC = protocolFeeBps == 0 ? 0 : FullMath.mulDiv(paidUSDC, protocolFeeBps,
    splitFeeBps);
```

In most compound and collect flows, the caller is also affected by slippage on rebalancing swaps or USDC conversion swaps, so they have a natural incentive to keep `slippageBps` reasonable. However, there is a specific scenario where the incentive misalignment becomes real:

When a position is out of range (or the collected fee ratio already matches the position's required liquidity ratio), the compound needs no rebalancing swap. The planner returns `token0ToToken1 = 0`, `token1ToToken0 = 0`, `_executeRebalance` exits at its first check (UniswapAdapter / AerodromeAdapter), and `addLiquidity` mints directly with the remaining tokens. In this case, the only swap in the entire compound operation is the fee slice sell, and its output goes entirely to the protocol. The caller's remaining tokens compound at full value regardless of how badly the fee swap executes.

A caller in this scenario can set `slippageBps` to a high value (up to 9999) with zero cost to themselves, then sandwich their own `compoundFees` transaction. The swap's `amountOutMinimum` on the router is derived from the caller's slippage budget (`_minOutFromLossBudget` in the adapter), so at high slippage it drops to near zero. The caller captures most of the fee slice as MEV profit while the protocol receives far less than the fair fee. Even without a deliberate self-sandwich, high slippage settings expose the fee swap to opportunistic MEV bots.

Note that deposit flow fees (`openPosition`, `addCollateral`) do not have this problem because they are computed on raw USDC input before any swap.

Recommendation

Cap the slippage tolerance used for protocol fee swaps independently of the user's overall slippage setting. A dedicated `maxProtocolFeeSlippageBps` (e.g. 1000–1500 bps) would prevent meaningful extraction while still handling normal volatility, without restricting users who have legitimate reasons to set high slippage on their own operations.

Team Response

Fixed: Added `maxProtocolFeeSlippageBps`, defaulting to 1_500 (15%). `compoundFees` and `changeRange` now cap slippage with `min(slippageBps, maxProtocolFeeSlippageBps)` when selling collected fee-token slices to fund earned-fee protocol fees.

Low/Info Findings

[L-01] Hostile ERC721 receiver can DoS emergency returnNft batches

Acknowledged

During a paused emergency, the protocol owner can call `returnNft(keys)` to batch-return position NFTs to their owners. For `EZWrapper` owned positions, `CLManagerUtils.returnNft` first asks `CORE` to return the `NFT` to `EZWrapper`, then calls `EZWrapper.creditReturnedNft` to forward it to the mapped user.

If the mapped user is a malicious contract that reverts in `onERC721Received`, the `safeTransferFrom` inside `creditReturnedNft` reverts. Because the call is not wrapped in `try/catch`, the revert bubbles up and reverts the entire batch, blocking benign users bundled in the same emergency return call.

```
1         address npm = ICLDexAdapter(p.dex).getNPM();
2         uint256 dustReturned = p.dustUSDC;
3
4         _debitActiveReferredUSDC(cfg, p, key, p.totalDepositedUSDC);
5         // CORE will transfer the NFT (and any tracked dust) to the owner
6         // and deregister the position.
7         cfg.CORE.returnPosition(key);
8         if (p.owner == address(cfg.EZ_WRAPPER) && msg.sender != address(cfg.EZ_WRAPPER
9             )) {
10             cfg.EZ_WRAPPER.creditReturnedNft(key, npm, p.tokenId, dustReturned);
11         }
```

- `creditReturnedNft` forwards the NFT with `safeTransferFrom`, which invokes the mapped user's ERC721 receiver hook:

```
1     function creditReturnedNft(bytes32 key, address npm, uint256 tokenId, uint256
2         dustReturned) external override {
3         if (msg.sender != CORE.manager()) revert NotManager();
4         if (npm == address(0) || tokenId == 0) revert InvalidReturnedNft();
5
6         address user = keyUser[key];
7         if (user == address(0)) revert NotPositionUser();
8
9         _removeStoredPosition(user, key);
10        INonfungiblePositionManager(npm).safeTransferFrom(address(this), user, tokenId);
11        if (dustReturned > 0) {
```

-A malicious `EZWrapper` user can cheaply poison emergency returnNft batches and delay recovery for benign users during an incident.

Recommendation

Wrap per-key NFT forwarding in `try/catch` and escrow failed forwards for later claim/rescue

Team Response

`returnNft` intentionally reverts if an `EZWrapper`-owned NFT cannot be forwarded to the mapped user. This avoids creating an intermediate state where the position is deregistered from `CLCore` but the NFT is stranded in `EZWrapper`.

Emergency returns are intended to be atomic per batch: positions either remain fully accounted for in `CLCore`, or are returned to the user path successfully. If one key reverts, operators can exclude that key and retry benign users in a separate batch.

[L-02] changeRange accepts identical ticks, charging bot fee and slippage for a no-op rebalance

Fixed

`CLManagerUtils.changeRange` validates only that `newTickLower < newTickUpper`, that the ticks align with `tickSpacing`, and that they fall within the global Uniswap V3 tick bounds. It does **not** check that the new range differs from the position's current range (`p.tickLower`, `p.tickUpper`).

If the function is called with `newTickLower == p.tickLower` and `newTickUpper == p.tickUpper`, the contract still:

1. Burns the existing NFT and unwinds it to the underlying tokens,
2. Re-mints a new NFT with the exact same ticks,
3. Pays the bot fee on the full reminted position value (`_payChangeRangeFees`)

The position owner loses value to the bot fee and slippage with zero rebalancing benefit. The most likely trigger is a backend bug (automation watcher mistakenly queues a rebalance job when no tick movement has occurred).

```
1 // CLManagerUtils.sol:660-668
2 slippageBps = slippageBps >= BPS ? BPS - 1 : slippageBps;
3 if (newTickLower >= newTickUpper) revert InvalidTickRange();
4
5 Position memory p = cfg.CORE.getPosition(key);
6 // ...
7 if ((newTickLower % p.tickSpacing) != 0 || (newTickUpper % p.tickSpacing) != 0) revert
   InvalidTickRange();
8 if (newTickLower < -887272 || newTickUpper > 887272) revert InvalidTickRange();
9 // no check that (newTickLower, newTickUpper) != (p.tickLower, p.tickUpper)
```

Recommendation:

Add a guard at the top of `changeRange` that rejects calls where the new range matches the current range. Also a bot blocker if `changeRange` was called in the last 5 minutes or so. It is generally good to be defended against some unpleasent backend off-chain issues and open doors for users to be harmed.

Team Response

Fixed: `changeRange` now reverts with `RangeUnchanged()` when the requested ticks match the position's current range.

[L-03] Referral fees become permanently stuck if referrer is USDC-blacklisted

Acknowledged

`ReferralManager.claimReferralFees` transfers USDC directly to `msg.sender`. USDC's (Circle/OFAC) maintains a blacklist. If a referrer's address is blacklisted, the `safeTransfer` call reverts permanently. The function provides no alternative recipient parameter, so the referrer's accumulated `referralBalances[msg.sender]` becomes unreachable and stranded in the contract.

```
1 function claimReferralFees(uint256 amount) external nonReentrant returns (uint256) {
2     if (amount == 0) revert ZeroAmount();
3
4     uint256 balance = referralBalances[msg.sender];
5     if (amount > balance) revert ZeroAmount();
```

```

6
7     referralBalances[msg.sender] = balance - amount;
8     USDC.safeTransfer(msg.sender, amount); // reverts if msg.sender is blacklisted
9
10    emit ReferralFeesClaimed(msg.sender, amount);
11    return amount;
12 }

```

Recommendation:

Allow the referrer to specify a recipient address for the payout. This also gives users flexibility to direct fees to a different address.

```

1  function claimReferralFees(uint256 amount, address to) external nonReentrant returns (
2      uint256) {
3      if (amount == 0) revert ZeroAmount();
4      if (to == address(0)) revert ZeroAddress();
5
6      uint256 balance = referralBalances[msg.sender];
7      if (amount > balance) revert ZeroAmount();
8
9      referralBalances[msg.sender] = balance - amount;
10     USDC.safeTransfer(to, amount);
11
12     emit ReferralFeesClaimed(msg.sender, to, amount);
13     return amount;
14 }

```

Team Response

Acknowledged: The system relies on USDC throughout core user flows, fee accounting, reserves, dust, and referral payouts, so issuer-level transfer restrictions are a broader external dependency rather than a referral-specific custody issue. This risk will be disclosed in user-facing terms/docs.

[L-04] Renounced ownership creates irremovable permanent pause authority**Fixed**

All three contracts (CLManager, ReferralManager, EZWrapper) inherit from OpenZeppelin's `Ownable`, which includes an inherited `renounceOwnership()` function that permanently sets `owner = address(0)`. None of the three contracts override or disable this function. After `renounceOwnership()` is called `owner` is permanently `address(0)`, so no owner-gated function can ever be called again.

Recommendation:

Override `renounceOwnership()` in all three contracts to revert unconditionally, preventing accidental or malicious ownership abandonment.

```

1  function renounceOwnership() public override onlyOwner {
2      revert("renouncing ownership not allowed");
3  }

```

Team Response

Fixed: CLManager, ReferralManager, and EZWrapper now override `renounceOwnership()` and revert with `OwnershipRenounceDisabled()`, preventing accidental or malicious ownership abandonment.

[L-05] Contracts Use Single-Step Ownable Instead of Ownable2Step**Acknowledged**

CLManager, ReferralManager, and EZWrapper inherit OpenZeppelin Ownable, which transfers ownership in a single transaction. If ownership is transferred to the wrong address, an unrecoverable address, or a contract unable to manage the protocol, privileged configuration can be permanently lost.

```
1
2  contract CLManager is ReentrancyGuard, Pausable, Ownable {
3
4  -----
5
6  contract ReferralManager is Ownable, Pausable, ReentrancyGuard {
7
8  -----
9
10 contract EZWrapper is Ownable, Pausable, ReentrancyGuard, IERC721Receiver, IEZWrapper {
```

- privileged ownership transfer uses a single-step handoff with no acceptance confirmation from the new owner.

Recommendation

replace Ownable with OpenZeppelin Ownable2Step on all three contracts. This requires the pending owner to explicitly accept ownership, reducing risk from mistyped or incompatible owner addresses.

Team Response

Acknowledged: A migration to Ownable2Step will be considered for a future version where the ownership pattern can be updated consistently across all system contracts.

[I-01] storeReferrer and resolveAndStoreReferrer are duplicate functions**Fixed**

ReferralManager.storeReferrer and ReferralManager.resolveAndStoreReferrer are identical: both are external, carry the same onlyManager whenNotPaused modifiers, take the same (address user, address candidate) parameters, return the same address referrer, and have the exact same body that simply forwards to the internal _storeReferrer(user, candidate).

```
1  function storeReferrer(address user, address candidate)
2      external
3      onlyManager
4      whenNotPaused
5      returns (address referrer)
6  {
7      referrer = _storeReferrer(user, candidate);
8  }
9
10 function resolveAndStoreReferrer(address user, address candidate)
11     external
12     onlyManager
13     whenNotPaused
14     returns (address referrer)
15 {
16     referrer = _storeReferrer(user, candidate);
17 }
```

Recommendation: Remove one of the two functions and update all callers and the interface to use the single remaining function.

Team Response

Fixed: Removed duplicate `resolveAndStoreReferrer` from `ReferralManager`. All manager call sites now use `storeReferrer`.

[I-02] Dead Self-Referrer Checks in CLManager

Fixed

`CLManager` checks whether the resolved referrer equals the effective user after calling `ReferralManager.resolveAndStoreReferrer`. These checks are unreachable because `ReferralManager._validReferrer` already rejects `referrer == user` before returning a referrer.

```
1      ReferralInfo memory resolvedReferrer;  
2      (resolvedReferrer.referrer, resolvedReferrer.shareBps) =  
3          REFERRAL_MANAGER.resolveAndStoreReferrer(effectiveUser, address(0), usdcAmount  
4          );  
5      if (resolvedReferrer.referrer == effectiveUser) {  
6          revert InvalidReferrer();  
      }
```

```
1      if (address(REFERRAL_MANAGER) == address(0)) revert ReferralManagerNotSet();  
2      (info.referrer, info.shareBps) = REFERRAL_MANAGER.resolveAndStoreReferrer(user,  
3          referrer, addedActiveUSDC);  
4      if (info.referrer == user) revert InvalidReferrer();
```

```
1      function _validReferrer(address user, address referrer) internal view returns (bool) {  
2          if (referrer == address(0)) return false;  
3          if (user != address(0) && referrer == user) return false;  
4          if (referrer == address(this) || referrer == manager) return false;  
5          return true;  
      }
```

The same invariant is enforced both in `ReferralManager` and again in `CLManager`, but the outer checks can never trigger if `ReferralManager` behaves as implemented.

Recommendation

remove the unreachable checks from `CLManager`, or replace them with a short comment documenting the invariant enforced by `ReferralManager`.

Team Response

Fixed: Removed the unreachable self-referral checks in `CLManager`.

[I-03] All 10 CLManager admin setter functions emit no events

Fixed

`CLManager` contains 10 `onlyOwner` admin setters (`setPlanner`, `setValuation`, `setGuardian`, `setEZWrapper`, `setReferralManager`, `setDiscountedWalletBps`, `setMinimumOpenUSDC`,

`setBotFeeMultiplierForEarnedFees`, `setEarnedFeesProtocolFeeBps`, `setMaxBatchKeys`) that modify critical infrastructure addresses and fee parameters without emitting any event. This is inconsistent with sibling contracts (ReferralManager emits `GuardianUpdated/ManagerUpdated/DefaultReferrerUpdated/MaxReferralShareBpsUpdated/ReferralTierUpdated`; EZWrapper emits `GuardianUpdated`; CLCore emits `BotFeeUpdated`).

Off-chain monitoring, governance tooling, indexers, and user position alerts cannot detect changes.

Recommendation:

Add a named event for each setter and emit it on every state change.

Team Response

Fixed: All CLManager owner-only admin setters now emit descriptive custom events for off-chain monitoring, aligning CLManager observability with other system contracts.

[I-04]validateActionReferrer Return Value Ignored in copyPosition**Fixed**

`copyPosition` calls `ReferralManager.validateActionReferrer(msg.sender, sourceOwner)` but discards both returned values. The call is therefore only useful if it reverts. However, `validateActionReferrer` resolves invalid candidates by falling back to `defaultReferrer`, so with a valid default referrer this call does not prove that `sourceOwner` itself is valid.

```
1 REFERRAL_MANAGER.validateActionReferrer(msg.sender, sourceOwner);
2 address storedReferrer = REFERRAL_MANAGER.storeReferrer(msg.sender, sourceOwner);
3 uint256 sourceTierAddedUSDC = storedReferrer == sourceOwner ? usdcAmount : 0;
4 uint16 referralShareBps = REFERRAL_MANAGER.referralShareBpsForAfter(sourceOwner,
    sourceTierAddedUSDC);
```

The code appears to validate `sourceOwner`, but ignores the canonical resolved referrer returned by `validateActionReferrer`.

Recommendation

I would suggest either remove the dead validation call, or enforce that the returned referrer equals `sourceOwner`.

```
1 (address validReferrer,) = REFERRAL_MANAGER.validateActionReferrer(msg.sender, sourceOwner);
2 if (validReferrer != sourceOwner) revert InvalidReferrer();
```

Team Response

Fixed: Removed the unused `validateActionReferrer` call from `copyPosition`.